

Θεωρία: Λίστες στην Python

Εισαγωγή στις Λίστες

Οι λίστες αποτελούν τύπο δεδομένων της Python.

Αντιστοιχίζουν μια σειρά δεδομένων σε μια μεταβλητή.

Όταν ορίζουμε μια λίστα, βάζουμε τα δεδομένα σε αγκύλες και τα χωρίζουμε με κόμματα.

Παράδειγμα:

```
my_list = ["yes", "no", "maybe"]
```

Τα δεδομένα εντός μιας λίστας μπορεί να ανήκουν σε οποιοδήποτε τύπο.

Παράδειγμα:

```
random_list = ["pancakes", 8988676767, 9.44, False]
```

- Ορίζουμε μια κενή λίστα, δηλαδή μια λίστα που δεν περιέχει στοιχεία, με αγκύλες [].

Παράδειγμα:

```
my_empty_list = []
```

Μέγεθος της λίστας – length

Ο αριθμός των στοιχείων μίας λίστας είναι το μέγεθός της. Μπορούμε να υπολογίσουμε αυτόν τον αριθμό χρησιμοποιώντας τη συνάρτηση **len()**.

Παράδειγμα:

```
fruit_basket = ["apple", "banana", "cherry"]  
print(len(fruit_basket))
```

Output

3

Προσπέλαση στοιχείων - Indexing

Μπορούμε να έχουμε πρόσβαση στα στοιχεία μίας λίστας χρησιμοποιώντας δείκτες (indexes) μαζί με το όνομα της λίστας.

Οι δείκτες είναι απλά ένα νούμερο που μας δείχνει την θέση του στοιχείου στη λίστα, η αρίθμηση αυτή ξεκινάει από το 0, όχι από το 1. Ας δούμε αναλυτικά για το προηγούμενο παράδειγμα λίστας που δημιουργήσαμε:

Indexes		
0	1	2
"apple"	"banana"	"cherry"

fruit_basket = ["apple", "banana", "cherry"]

 **ΠΡΟΣΟΧΗ:** Η αρίθμηση ξεκινά από το μηδέν!

Προσοχή, βλέπουμε ότι οι δείκτες ξεκινάνε από το 0 άρα αυτό που βλέπουμε ως δεύτερο στοιχείο στην λίστα θα έχει index 1, το τρίτο 2 κλπ, δηλαδή ένα μικρότερο από αυτό που αυθόρμητα θα σκεφτόμασταν.

Άρα για να έχουμε πρόσβαση στα στοιχεία της λίστας "apple", "banana", "cherry" χρειάζεται να χρησιμοποιήσουμε τους δείκτες τους μέσα στην λίστα:

- "apple" `fruit_basket[0]`
 - "banana" `fruit_basket[1]`
 - "cherry" `fruit_basket[2]`
- Τα δεδομένα είναι αριθμημένα. Μπορούμε να αποκτήσουμε πρόσβαση σε αυτά μέσω του αριθμητικού τους "δείκτη", γράφοντας το όνομα της λίστας και τον αριθμό του στοιχείου

Πρόγραμμα	Έξοδος
<code>print(random_list)</code>	<code>['pancakes', 8988676767, 9.44, False]</code>
<code>print(random_list[0])</code>	<code>pancakes</code>
<code>my_favorite_number = random_list[2]</code> <code>print(my_favorite_number)</code>	<code>9.44</code>

Αρνητικοί δείκτες

Μπορούμε να χρησιμοποιήσουμε αρνητικούς δείκτες για να προσπελάσουμε τα στοιχεία της λίστας από το τέλος της λίστας:

		Indexes		
	-3	-2	-1	
	"apple"	"banana"	"cherry"	

`fruit_basket = ["apple", "banana", "cherry"]`

- "cherry" `fruit_basket[-1]` // 1ο στοιχείο από το τέλος
 - "banana" `fruit_basket[-2]` // 2ο στοιχείο από το τέλος
 - "apple" `fruit_basket[-3]` // 3ο στοιχείο από το τέλος
- Τα στοιχεία αριθμούνται και από το τέλος προς την αρχή.

Πρόγραμμα	Έξοδος
<code>print(random_list[-2])</code>	<code>9.44</code>

Προσπέλαση στοιχείων - Slicing

Μπορούμε να έχουμε πρόσβαση σε ολόκληρο τμήμα της λίστας χρησιμοποιώντας τους δείκτες.

Σύνταξη

`my_list[start:end:step]`

- **start**: από ποια θέση ξεκινάμε
- **end**: έως ποια θέση (δεν συμπεριλαμβάνεται στην υπολίστα που επιστρέφεται!)
- **step**: πόσα βήματα να κάνουμε για το επόμενο στοιχείο

Αν παραλείψεις κάποιο από αυτά, παίρνει προεπιλεγμένη τιμή.

- ✓ Αν αφήσουμε το start κενό → ξεκινάει από την αρχή της λίστας

- ✓ Αν αφήσουμε το end κενό → φτάνει έως το τέλος
- ✓ Αν αφήσουμε και τα δύο κενά: `my_list[:]` → παίρνουμε ολόκληρη τη λίστα.
- ✓ Αν αφήσουμε το step κενό → παίρνει την τιμή 1, δηλαδή επιστρέφει συνεχόμενα στοιχεία
- ✓ Μπορούμε επίσης να χρησιμοποιούμε αρνητικούς δείκτες (π.χ. -1, -2) για να μετράμε από το τέλος.

Παραδείγματα

Indexes					
0	1	2	3	4	5
1	2	3	4	5	6

`nums = [1, 2, 3, 4, 5, 6]`

Παράδειγμα 1 — Από θέση start έως end:

```
nums = [1, 2, 3, 4, 5, 6]
print(nums[2:5])    # από το στοιχείο στην θέση 2 έως πριν τη θέση 5 (2 : 5 : _)
```

Output:

```
2  3  4
[3, 4, 5]
```

i Εδώ βλέπουμε πως ενώ επιλέξαμε `end=5` το στοιχείο στη θέση 5 (`nums[5]` → 6) δεν επιστρέφεται, το τελευταίο στοιχείο που επιστρέφεται είναι αυτό στη θέση 4 (`nums[4]` → 5), γιατί το end έως ποια θέση, αυτή η θέση δεν επιστρέφεται, φτάνουμε μέχρι την προηγούμενη.

Παράδειγμα 2 — Από την αρχή έως θέση end:

```
nums = [1, 2, 3, 4, 5, 6]
print(nums[:4])    # παραλείψαμε αρχή και step (_ : 4 : _)
```

Output:

```
0  1  2  3
[1, 2, 3, 4]    # επιστρέφουμε από θέση 0 έως και την θέση 3
```

Παράδειγμα 3 — Από θέση start έως το τέλος:

```
nums = [1, 2, 3, 4, 5, 6]
print(nums[3:])    # παραλείψαμε τέλος και step (3:_:_)
```

Output:

```
3  4  5
[4, 5, 6]
```

Παράδειγμα 4 — Χρήση step (βήμα):

```
nums = [1, 2, 3, 4, 5, 6]
print(nums[::2])    # από αρχή έως τέλος παίρνω κάθε 2ο στοιχείο
```

Output:

```
0  2  4
[1, 3, 5]
```

Παράδειγμα 5 — Αρνητικοί δείκτες:

```
nums = [1, 2, 3, 4, 5, 6]
print(nums[-3:-1])    # από τέλος έως τρίτο στοιχείο από το τέλος
```

Output:

```
-3  -2
[4, 5]
```

i Εδώ βλέπουμε πως και για τους αρνητικούς δείκτες, δεν επιστρέφουμε την θέση -1 (τελευταίο στοιχείο) αλλά μια θέση πριν από αυτήν (-2).

Παράδειγμα 6 — Αντιστροφή λίστας με slicing:

```
nums = [1, 2, 3, 4, 5, 6]
print(nums[::-1])    # (_::-1)
```

Output:

```
-1 -2 -3 -4 -5 -6
[6, 5, 4, 3, 2, 1]
```

i Όταν χρησιμοποιούμε αρνητικό step χρησιμοποιούμε το αντίστοιχο θετικό step (π.χ. 1) απλά ξεκινώντας από το τέλος της λίστας.

Έλεγχος αν ένα στοιχείο υπάρχει στη λίστα

Μπορούμε να χρησιμοποιήσουμε το keyword **in** για να ελέγξουμε εάν κάποιο συγκεκριμένο στοιχείο περιέχεται σε μια λίστα.

Παράδειγμα:

```
fruit_basket = ["apple", "banana", "cherry"]
if "apple" in fruit_basket:
    print("Yes, 'apple' is in the fruits list")
```

i Με το keyword **in** δημιουργούμε μια συνθήκη που μπορούμε να τοποθετήσουμε μέσα στο **if** statement.

Αλλαγή στοιχείων μιας λίστας

Στις λίστες μπορούμε να τροποποιήσουμε στοιχεία, χρησιμοποιώντας το **index** του στοιχείου και τον τελεστή της ανάθεσης (**=**).

Παράδειγμα:

```
fruit_basket = ["apple", "banana", "cherry"]
fruit_basket[1] = "strawberry" # αλλαγή του στοιχείου στη θέση 1 σε "strawberry"
                                # "banana" ➡ "strawberry"
print(fruit_basket)
```

Output

```
['apple', 'strawberry', 'cherry']
```

Μπορούμε επίσης να αλλάξουμε περισσότερα από ένα στοιχεία ταυτόχρονα, χρησιμοποιώντας slicing.

Παράδειγμα:

Indexes					
0	1	2	3	4	5
"apple"	"banana"	"cherry"	"orange"	"kiwi"	"mango"

```
fruit_basket = ["apple", "banana", "cherry", "orange", "kiwi", "mango"]
fruit_basket[1:3] = ["strawberry", "watermelon"] # αλλαγή των στοιχείων 1 έως 3 (1,2)
print(fruit_basket)
```

Output

```
['apple', 'strawberry', 'watermelon', 'orange', 'kiwi', 'mango']
```

! Προσοχή, τα νέα στοιχεία "strawberry" και "watermelon" τα δίνουμε σε μορφή λίστας: ["strawberry", "watermelon"]

Εάν το διάστημα της λίστας που ορίσαμε είναι μικρότερο από τα στοιχεία που δίνουμε:

Indexes		
0	1	2
"apple"	"banana"	"cherry"

```
fruit_basket = ["apple", "banana", "cherry"]
fruit_basket[1:2] = ["strawberry", "watermelon"]
print(fruit_basket)
```

Output

```
['apple', 'strawberry', 'watermelon', 'cherry']
```

Δίνουμε διάστημα ενός στοιχείου στην λίστα (θέση 1) και 2 νέα στοιχεία. Η Python στη θέση που ορίσαμε (θέση 1) θα κάνει τις αλλαγές "banana" "strawberry" και ύστερα θα προσθέσει τα περισσευούμενα στοιχεία "watermelon". Τα στοιχεία που ήταν εκτός του διαστήματος αλλαγής θα μετακινηθούν αντίστοιχα αλλά δεν θα αλλάξουν.

```
['apple', 'strawberry', 'watermelon', 'cherry']
θέση 0          νέο!          θέση 3
```

Εάν το διάστημα της λίστας που ορίσαμε είναι μεγαλύτερο από τα στοιχεία που δίνουμε:

Indexes		
0	1	2
"apple"	"banana"	"cherry"

```
fruit_basket = ["apple", "banana", "cherry"]  
fruit_basket[1:3] = ["watermelon"]  
  
print(fruit_basket)
```

Output
['apple', 'watermelon']

Δίνουμε διάστημα δύο στοιχείων στην λίστα (θέση 1 και 2) και 1 νέο στοιχείο. Η Python θα διαγράψει τα στοιχεία για τα οποία δεν έχουμε ορίσει την νέα τιμή. Συγκεκριμένα, στη θέση 1 θα κάνει την αλλαγή "banana" "watermelon", αφού υπάρχει η νέα τιμή, και ύστερα θα διαγράψει τη λέξη στη θέση 2 για την οποία δεν έχουμε νέα τιμή.

```
['apple', 'banana', 'cherry'] -> ['apple', 'watermelon']
```

⊕ Εισαγωγή νέων στοιχείων σε λίστα

Append()

Χρησιμοποιείται για να προσθέσουμε ένα νέο στοιχείο στο τέλος της λίστας.

Σύνταξη

```
fruit_basket.append(new_item)
```

Μέσα στην παρένθεση βάζουμε το νέο στοιχείο που θέλουμε να προστεθεί.

Παράδειγμα:

```
fruit_basket = ["apple", "banana", "cherry"]  
fruit_basket.append("orange")  
  
print(fruit_basket)
```

Output
['apple', 'banana', 'cherry', 'orange']

⚠ Δεν μπορούμε να χρησιμοποιήσουμε την συνάρτηση append() για να προσθέσουμε παραπάνω από ένα στοιχείο στην λίστα: `fruit_basket.append("orange", "lemon")`

Insert()

Χρησιμοποιείται για να προσθέσουμε ένα νέο στοιχείο σε συγκεκριμένη θέση (index).

Σύνταξη

```
fruit_basket.insert(index, new_item)
```

Μέσα στην παρένθεση δίνουμε δύο ορίσματα, πρώτο το index στο οποίο θα προστεθεί το νέο στοιχείο και δεύτερο το νέο στοιχείο που θέλουμε να προστεθεί.

Παράδειγμα:

```
fruit_basket = ["apple", "banana", "cherry"]
fruit_basket.insert(1, "orange")

print(fruit_basket)
```

Output

```
0      1      2      3
['apple', 'orange', 'banana', 'cherry']
```

! Δεν μπορούμε να χρησιμοποιήσουμε την συνάρτηση insert() για να προσθέσουμε παραπάνω από ένα στοιχείο στην λίστα: `fruit_basket.insert(1, "orange", "lemon")`

Extend()

Η συνάρτηση extend() χρησιμοποιείται όταν θέλουμε να προσθέσουμε πολλά στοιχεία σε μια λίστα με μία μόνο κίνηση.

Πρακτικά, κάνει append κάθε στοιχείο από μια άλλη λίστα

Σύνταξη

```
fruit_basket. extend(new_list)
```

Μέσα στην παρένθεση βάζουμε το νέα λίστα που θέλουμε να προστεθεί.

Παράδειγμα:

```
fruit_basket = ["apple", "banana", "cherry"]
newlist = ["mango", "pineapple", "papaya"]
fruit_basket.extend(newlist)

print(fruit_basket)
```

Output

```
['apple', 'banana', 'cherry', 'mango', 'pineapple', 'papaya']
```

Το ίδιο ακριβώς μπορούμε να κάνουμε χρησιμοποιώντας τον τελεστή '+', για να προσθέσουμε δύο λίστες.

```
fruit_basket = ["apple", "banana", "cherry"]
newlist = ["mango", "pineapple", "papaya"]
updatedlist = fruit_basket + newlist

print(updatedlist)
```

Output

```
['apple', 'banana', 'cherry', 'mango', 'pineapple', 'papaya']
```



Διαγραφή στοιχείων από λίστα

remove()

Η μέθοδος remove() αφαιρεί το πρώτο στοιχείο της λίστας που έχει την τιμή που της δίνουμε.

Σύνταξη

```
fruit_basket.remove(item)
```

Μέσα στην παρένθεση βάζουμε το στοιχείο που θέλουμε να διαγραφεί.

Παράδειγμα:

```
fruit_basket = ["apple", "banana", "cherry"]
fruit_basket.remove("banana")

print(fruit_basket)
```

Output

```
['apple', 'cherry']
```



Αν υπάρχουν πολλαπλές εμφανίσεις του ίδιου στοιχείου, η remove() αφαιρεί μόνο την πρώτη.

Παράδειγμα:

```
fruit_basket = ["apple", "banana", "cherry", "banana", "kiwi"]
fruit_basket.remove("banana")
print(fruit_basket)
```

Output

```
['apple', 'cherry', 'banana', 'kiwi']
```

Το 2ο στοιχείο "banana" παρέμεινε στην τελική λίστα.

pop()

Η μέθοδος pop() αφαιρεί το στοιχείο στη θέση (index) που της δίνουμε.

Σύνταξη

```
fruit_basket.pop(index)
```

Μέσα στην παρένθεση βάζουμε το index του στοιχείου που θέλουμε να διαγραφεί.

Παράδειγμα:

```
fruit_basket = ["apple", "banana", "cherry"]
fruit_basket.pop(1) # διαγραφή του fruit_basket[1]

print(fruit_basket)
```

Output

```
['apple', 'cherry']
```



Αν δεν δώσουμε index, η pop() αφαιρεί το τελευταίο στοιχείο της λίστας.

Παράδειγμα:

```
fruit_basket = ["apple", "banana", "cherry"]
fruit_basket.pop()

print(fruit_basket)
```

Output

```
['apple', 'banana']
```

del

Το keyword `del` μπορεί να χρησιμοποιηθεί για να διαγράψει ένα στοιχείο σε συγκεκριμένη θέση ή ακόμη και ολόκληρη τη λίστα.

Σύνταξη

```
del fruit_basket[index]    # διαγραφή ενός στοιχείου της λίστας
del fruit_basket           # διαγραφή όλης της λίστας
```

Παράδειγμα:

```
fruit_basket = ["apple", "banana", "cherry"]
del fruit_basket[0]    # διαγραφή του fruit_basket[0]

print(fruit_basket)
```

Output

```
['banana', 'cherry']
```

clear()

Η μέθοδος `clear()` αδειάζει τη λίστα, αφήνοντάς την κενή, αλλά χωρίς να τη διαγράψει. Δηλαδή, σε αντίθεση με την εντολή `del`, μετά το `clear` μπορώ να προσθέσω νέα στοιχεία στην λίστα (π.χ. με `append()`).

Σύνταξη

```
fruit_basket.clear()
```

Παράδειγμα:

```
fruit_basket = ["apple", "banana", "cherry"]
fruit_basket.clear()

print(fruit_basket)
```

Output

```
[]
```

Η λίστα εξακολουθεί να υπάρχει, αλλά δεν έχει πια περιεχόμενο.



Αντιγραφή λίστας

Δεν μπορούμε να αντιγράψουμε το περιεχόμενο της λίστας `fruit_basket` στην `fruit_basket2` γράφοντας:

```
fruit_basket2 = fruit_basket
```

Με αυτή τη γραμμή δεν δημιουργούμε νέα λίστα. Αντίθετα, δημιουργούμε ένα δεύτερο όνομα (reference) που δείχνει στην ίδια λίστα στη μνήμη.

Αυτό σημαίνει ότι:

- ✓ οι δύο μεταβλητές δείχνουν στο ίδιο αντικείμενο
- ✓ οποιαδήποτε αλλαγή κάνουμε στη μία λίστα θα εμφανιστεί και στην άλλη
- ✓ οι λίστες δεν είναι πλέον ανεξάρτητες μεταξύ τους

Αυτό συνήθως δεν είναι επιθυμητό, γιατί θέλουμε η κάθε λίστα να μπορεί να αλλάζει χωρίς να επηρεάζει την άλλη.

Για να αντιγράψουμε το περιεχόμενο μιας λίστας χρησιμοποιούμε την συνάρτηση `copy()`.

Σύνταξη

```
newlist = oldlist.copy()
```

Παράδειγμα:

```
fruit_basket = ["apple", "banana", "cherry"]  
fruit_basket2 = fruit_basket.copy()  
print(fruit_basket2)
```

Output

```
['apple', 'banana', 'cherry']
```



Επανάληψη (Loop) μέσα από μια λίστα

Η επανάληψη είναι πολύ χρήσιμη όταν θέλουμε να επεξεργαστούμε ή να εμφανίσουμε κάθε στοιχείο της λίστας.

Loop με `for` – Εμφάνιση στοιχείων ένα προς ένα

Μπορούμε να χρησιμοποιήσουμε `for loop` για να προσπελάσουμε τα στοιχεία μια λίστας.

Παράδειγμα:

```
fruit_basket = ["apple", "banana", "cherry"]  
for x in fruit_basket:  
    print(x)
```

Output

```
apple  
banana  
cherry
```

Αυτό, όπως βλέπουμε, θα εμφανίσει κάθε στοιχείο της λίστας σε ξεχωριστή γραμμή.

Μπορούμε την παραπάνω εντολή να την γράψουμε σε μια γραμμή ως εξής:

```
fruit_basket = ["apple", "banana", "cherry"]  
[print(x) for x in fruit_basket]
```

Output

```
apple  
banana  
cherry
```

Loop με δείκτες (index numbers)

Μπορούμε να κάνουμε το ίδιο πράγμα χρησιμοποιώντας τα indexes της λίστας για να προσπελάσουμε τα στοιχεία της. Θα χρησιμοποιήσουμε τις συναρτήσεις `range()` και `len()`.

Παράδειγμα:

```
fruit_basket = ["apple", "banana", "cherry"]
for i in range(len(fruit_basket)):
    print(fruit_basket[i])
```

Output

```
apple
banana
cherry
```

Στην ουσία εδώ κάνουμε μια προσπέλαση με indexes π.χ., `print(fruit_basket[0])` και για να βρούμε τον αριθμό των επαναλήψεων που θα εκτελέσουμε και θα τοποθετήσουμε μέσα στο `range` χρησιμοποιούμε την συνάρτηση `len()`. Άρα το `range(len(fruit_basket))` θα επιστρέψει τιμές από το 0 έως και το `len(fruit_basket)-1` που είναι όλα τα indexes της λίστας άρα θα την προσπελάσει ολόκληρη.

Loop με while

Αντίστοιχα, για την προσπέλαση μια λίστας μέσω indexes μπορούμε να χρησιμοποιήσουμε `while` loop.

Παράδειγμα:

```
fruit_basket = ["apple", "banana", "cherry"]
i = 0
while i < len(fruit_basket):
    print(fruit_basket[i])
    i = i + 1 # αύξηση του index, αλλιώς θα έχουμε infinite loop
```

Output

```
apple
banana
cherry
```

Το `while` σε αυτή την περίπτωση κάνει το ίδιο πράγμα με το `for`, αλλά μας δίνει περισσότερο έλεγχο.

↕ Ταξινόμηση λίστας

Η Python μας επιτρέπει να ταξινομούμε (να βάζουμε σε σειρά) τα στοιχεία μιας λίστας πολύ εύκολα χρησιμοποιώντας τη μέθοδο `sort()`.

Η ταξινόμηση γίνεται αλφαβητικά για λέξεις (strings) και αριθμητικά για αριθμούς.

i Υπενθύμιση:

Αύξουσα σειρά ταξινόμησης: Τα στοιχεία διατάσσονται από το μικρότερο στο μεγαλύτερο, δηλαδή κάθε επόμενο στοιχείο είναι μεγαλύτερο ή ίσο από το προηγούμενό του.

```
item1 < item2 < item3 < item4 < item5 < item6
```

π.χ. 2 3 6 9 17 22 33

Φθίνουσα σειρά ταξινόμησης: Τα στοιχεία διατάσσονται από το μεγαλύτερο στο μικρότερο, δηλαδή κάθε επόμενο στοιχείο είναι μικρότερο ή ίσο από το προηγούμενό του.

```
item1 > item2 > item3 > item4 > item5 > item6
```

π.χ. 33 22 17 9 6 3 2

sort() – Αύξουσα σειρά (default)

Η μέθοδος `sort()` ταξινομεί τη λίστα από το μικρότερο προς το μεγαλύτερο (ή αλφαβητικά από το A → Ω).

Παράδειγμα – Αλφαβητική ταξινόμηση

```
fruit_basket = ["orange", "mango", "kiwi", "pineapple", "banana"]
fruit_basket.sort()
print(fruit_basket)
```

Output

```
['banana', 'kiwi', 'mango', 'orange', 'pineapple']
```

Παράδειγμα – Αριθμητική ταξινόμηση

```
mylist = [100, 50, 65, 82, 23]
mylist.sort()
print(mylist)
```

Output

```
[23, 50, 65, 82, 100]
```

sort(reverse = True) – Φθίνουσα σειρά

Για να ταξινομήσουμε τη λίστα αντίστροφα (μεγαλύτερο → μικρότερο), χρησιμοποιούμε το όρισμα: `sort(reverse = True)`

Παράδειγμα – αλφαβητικά

```
fruit_basket = ["orange", "mango", "kiwi", "pineapple", "banana"]
fruit_basket.sort(reverse = True)
print(fruit_basket)
```

Output

```
['pineapple', 'orange', 'mango', 'kiwi', 'banana']
```

Παράδειγμα – αριθμητικά

```
mylist = [100, 50, 65, 82, 23]
mylist.sort(reverse = True)
print(mylist)
```

Output

```
[100, 82, 65, 50, 23]
```

T Μπορούμε να χειριζόμαστε strings ως λίστες!

Κώδικας	Αποτέλεσμα
<pre>word = "PYTHON" print(word[0])</pre>	P
<pre>print(word[-1])</pre>	N
<pre>word_length= len(word)</pre>	word_length = 6.
<pre>print(word[0:2])</pre>	PY
<pre>for letter in word: print(letter)</pre>	P Y T H O N
<pre>for i in range(len(word)): print(word[i])</pre>	P Y T H O N
<pre>lexi[0] = "C"</pre>	Δεν επιτρέπεται να αλλάξουμε γράμμα. Πρέπει να φτιάξουμε νέο string.

Extras - πως τοποθετώ input σε λίστα

Πολύ συχνά θέλουμε ο χρήστης να δώσει πολλά στοιχεία (αριθμούς ή λέξεις) μέσα σε μία γραμμή, χωρισμένα με κενά.

Η Python μας επιτρέπει να τα μετατρέψουμε εύκολα σε λίστα χρησιμοποιώντας την εντολή `split()`.

1. Διάβασμα λίστας από τον χρήστη (λέξεις ή strings)

Αν ο χρήστης γράψει:

```
apple banana cherry
```

```
fruits = input("Δώσε φρούτα χωρισμένα με κενό: ").split()
print(fruits)
```

Output

```
['apple', 'banana', 'cherry']
```

i Η μέθοδος `split()` «σπάει» το κείμενο σε λίστα χρησιμοποιώντας τα κενά.

2. Διάβασμα λίστας αριθμών

Όταν θέλουμε αριθμούς, πρέπει πρώτα να τους «σπάσουμε» με `split()` και μετά να τους μετατρέψουμε σε ακέραιους (`int`):

```
numbers = input("Δώσε αριθμούς χωρισμένους με κενό: ").split()
numbers = [int(x) for x in numbers]
```

Τι κάνει αυτή η γραμμή;

```
[int(x) for x in numbers]
```

- περνάει κάθε στοιχείο `x` της λίστας
- το μετατρέπει σε αριθμό `int`
- και φτιάχνει νέα λίστα με τους αριθμούς

Γενικός κανόνας

Για λέξεις:

```
mylist = input("Δώσε στοιχεία: ").split()
```

Για αριθμούς:

```
mylist = input("Δώσε αριθμούς: ").split()
mylist = [int(x) for x in mylist]
```